MERN STACK DEVELOPMENT PLAN

OmniChannel E-Commerce Platform

A comprehensive development blueprint for a production-grade, full-stack e-commerce and inventory management system built with MongoDB, Express.js, React, and Node.js.

STACK & SCOPE

14 MongoDB Collections | 50+ API Endpoints | 10 Core Features
Redis, Socket.io, Stripe, Docker | 6-Phase 12-Week Roadmap

AUGUST 2026 | PLACEMENT PORTFOLIO PROJECT

Table of Contents

1. Executive Summary

The OmniChannel E-Commerce and Inventory Management Platform is a production-grade, full-stack web application built on the MERN stack (MongoDB, Express.js, React, Node.js) augmented with Redis for session management and caching, Socket.io for real-time communication, and Stripe for payment processing. This project is designed to serve as the centerpiece of a software engineering placement portfolio, demonstrating mastery of distributed transaction management, role-based access control, real-time order tracking, and third-party API integration. The platform addresses the complexities of modern multi-vendor e-commerce by synchronizing inventory across multiple warehouses, preventing race conditions during flash sales, and providing a seamless shopping experience with real-time status updates.

Unlike standard CRUD applications that merely demonstrate basic database operations, this platform tackles real-world engineering challenges that interviewers actively evaluate: ACID-compliant transactions using MongoDB sessions, optimistic concurrency control for stock reservation, secure JWT-based authentication with HTTP-only cookie storage and refresh token rotation, and robust webhook verification for asynchronous payment confirmation. Each of the ten core features has been selected to showcase a distinct technical competency, from complex aggregation pipelines for faceted product search to background cron jobs for automated warehouse fulfillment monitoring. The architecture is designed to be horizontally scalable, containerized with Docker, and deployable to cloud platforms like Render or AWS.

This development plan provides a comprehensive blueprint covering system architecture, database schema design with complete field-level specifications for all MongoDB collections, REST API endpoint documentation with request/response payloads, and a phased development timeline with weekly milestones. The plan prioritizes features based on dependency ordering, beginning with authentication and catalog management as foundational layers, then building up to payment integration, real-time tracking, and analytics. By following this roadmap, a developer can systematically construct a platform that not only functions end-to-end but also demonstrates the architectural maturity and production-readiness that hiring managers expect from placement candidates.

Metric	Detail
Tech Stack	MongoDB, Express.js, React (Vite), Node.js, Redis, Socket.io, Stripe, PayPal, SendGrid, AWS S3
Total Features	10 production-grade modules covering auth, catalog, inventory, payments, tracking, fulfillment, coupons, reviews, analytics, and invoicing
Core Collections	14 MongoDB collections with full indexing, compound indexes, and text search indexes
API Endpoints	50+ RESTful routes across 6 resource controllers with Zod validation
Real-Time Channels	3 Socket.io event namespaces (orders, inventory, admin notifications)

Development Timeline	6 phases over 12 weeks with weekly milestones and deliverables

Table 1. Project Overview at a Glance

2. System Architecture

2.1 High-Level Architecture

The platform follows a classic three-tier architecture pattern with clear separation between the presentation layer, business logic layer, and data access layer. The React frontend communicates exclusively with the Express.js backend through a RESTful API gateway, while real-time features leverage a dedicated Socket.io server that runs alongside the HTTP server on the same Node.js process. Redis serves a dual purpose: it acts as a session store for JWT refresh tokens and as a caching layer for frequently accessed product catalog data, reducing MongoDB query load by approximately 60-70% for read-heavy catalog browsing operations. The entire application is containerized using Docker, with separate containers for the frontend (served by Nginx in production), the backend API server, MongoDB (with replica set configuration for transaction support), and Redis.

The frontend is built with React 18 using Vite as the build tool, Redux Toolkit for centralized state management, and Tailwind CSS for utility-first styling. React Router v6 handles client-side routing with protected route wrappers that redirect unauthenticated users to the login page. The backend follows a modular Express.js structure with separate route files for each resource domain (auth, products, orders, inventory, coupons, reviews), a centralized error-handling middleware, and a request validation layer using Zod schemas. MongoDB is accessed through Mongoose ODM with pre-defined models and validators. Environment variables are managed through a dotenv configuration file, with separate .env files for development, testing, and production environments.

Layer	Technology	Responsibility
Presentation	React 18 + Vite, Redux Toolkit, Tailwind CSS, React Router v6	UI rendering, client-side routing, state management, Socket.io client
API Gateway	Express.js, Zod validation, CORS, helmet, express-rate-limit	RESTful endpoints, input validation, JWT auth middleware, rate limiting
Real-Time	Socket.io, Redis Pub/Sub adapter	Live order tracking, inventory sync, admin alerts across clients
Business Logic	Node.js service layer, Mongoose models, node-cron	Transactions, discount calculation, stock reservation, PDF generation
Data Layer	MongoDB (replica set), Mongoose ODM, Redis	Persistent storage, text indexes, aggregation pipelines, session cache

External APIs	Stripe, PayPal, SendGrid, AWS S3	Payment processing, email notifications, image storage
DevOps	Docker, Docker Compose, Nginx	Container orchestration, reverse proxy, production deployment

Table 2. *Architecture Layer Breakdown*

2.2 Project Folder Structure

A clean, modular folder structure is critical for maintainability and is one of the first things interviewers evaluate when reviewing a codebase. The project follows a monorepo pattern with separate client and server directories at the root level, along with shared configuration files for Docker, environment variables, and documentation. Each feature domain within the server has its own directory containing route handlers, controller logic, service layer, validation schemas, and Mongoose models. This separation ensures that adding new features or modifying existing ones does not create side effects in unrelated modules, and it makes unit testing straightforward since each layer can be tested in isolation.

```
omnichannel-ecommerce/
|-- client/ # React frontend (Vite)
| |-- src/
| | |-- components/ # Reusable UI components
| | |-- pages/ # Route-level page components
| | |-- store/ # Redux Toolkit slices
| | |-- hooks/ # Custom React hooks
| | |-- services/ # Axios API client instances
| | |-- utils/ # Helpers, formatters, constants
|-- server/
| |-- config/ # DB, Redis, Stripe, env config
| |-- models/ # Mongoose schemas (14 models)
| |-- routes/ # Express route definitions
| |-- controllers/ # Request handlers
| |-- services/ # Business logic layer
| |-- middlewares/ # Auth, error, upload, RBAC
| |-- validators/ # Zod schemas
| |-- utils/ # PDF gen, email, helpers
| |-- server.js
|-- docker-compose.yml
|-- Dockerfile.client
|-- Dockerfile.server
|-- .env.example
|-- README.md
```

3. Database Schema Design

The database schema is designed around 14 core MongoDB collections that model the complete e-commerce domain. MongoDB was chosen for its flexible document model, which accommodates the varied and evolving product attribute structures inherent in multi-vendor catalogs. Unlike relational databases that require complex joins and ALTER TABLE operations for schema changes, MongoDB allows embedding related data within a single document and using atomic update operators for efficient

modifications. The schema leverages Mongoose validators for data integrity at the application level, MongoDB text indexes for full-text search performance, and compound indexes for common query patterns such as category-plus-price-range filtering. All monetary values are stored as integers representing the smallest currency unit (e.g., cents for USD) to avoid floating-point precision errors in financial calculations.

3.1 Users Collection

The Users collection stores all user accounts across the four role tiers: Super Admin, Vendor Manager, Warehouse Staff, and Customer. Each user document contains authentication credentials, profile information, and role-specific metadata. Passwords are hashed using bcrypt with a salt round of 12 before storage, and the password field is excluded from all API responses through Mongoose schema-level select:false configuration. The role field uses an enum constraint to prevent invalid role assignment, and the isActive flag enables soft deactivation of accounts without data deletion. Vendor Manager documents include an additional vendorId reference that links them to their associated vendor profile, while Warehouse Staff documents reference their assigned warehouse location.

Field	Type	Constraints	Description
name	String	required, trim	Full name of the user
email	String	required, unique, lowercase	Login email used as username
password	String	required, select:false	bcrypt hash with 12 salt rounds
role	String (enum)	required	super_admin | vendor_manager | warehouse_staff | customer
avatar	String (URL)	optional	S3 URL for profile picture
phone	String	optional	Phone number with country code
isActive	Boolean	default: true	Soft account deactivation flag
vendorId	ObjectId	ref: Vendor (conditional)	Link to vendor for vendor_manager role
warehouseId	ObjectId	ref: Warehouse (conditional)	Link to warehouse for warehouse_staff role
timestamps	Date	auto (Mongoose)	createdAt, updatedAt

Table 3. *Users Collection Schema*

3.2 Products Collection

The Products collection is the central entity of the catalog system. Each product document stores comprehensive information including a denormalized vendor name for fast display rendering, an array of variant objects for size and color options, and a flexible attributes map that accommodates category-specific metadata (e.g., material and fit for clothing, or storage capacity and processor type for electronics). The category and subcategory fields use string references rather than ObjectId references to simplify aggregation pipelines for faceted filtering. A compound text index on name, description, and brand enables high-performance full-text search with relevance scoring, while individual indexes on category, price, averageRating, and stockStatus support the faceted filtering pipeline that powers the product catalog browsing experience.

Field	Type	Constraints	Description
name	String	required, text index	Product title, indexed for search
description	String	required, text index	Detailed description, searchable
brand	String	text index	Brand name, part of search index
category	String	required, indexed	Top-level category (e.g., Electronics)
subcategory	String	indexed	Sub-category (e.g., Smartphones)
basePrice	Number (int)	required, min: 0	Price in cents (smallest currency unit)
salePrice	Number (int)	optional, min: 0	Discounted price in cents
images	[String]	required, max: 10	Array of S3 image URLs
variants	[Embedded Doc]	optional	Array of {size, color, sku, priceModifier}
attributes	Map (Object)	flexible	Category-specific key-value metadata
vendorId	ObjectId	ref: Vendor, indexed	Owning vendor reference
vendorName	String	denormalized	Cached vendor name for fast display
averageRating	Number	default: 0, indexed	Computed from reviews aggregation
stockStatus	String (enum)	indexed	in_stock | low_stock | out_of_stock
isActive	Boolean	default: true, indexed	Product listing visibility flag

Table 4. *Products Collection Schema*

3.3 Orders Collection

The Orders collection captures the complete lifecycle of a customer purchase, from initial creation through payment confirmation to warehouse fulfillment and final delivery. Each order document embeds an array of line items, each containing product details snapshot at the time of purchase (price, name, variant) to preserve historical accuracy even if the product catalog changes later. The order status field follows a strict state machine pipeline: pending_payment, processing, confirmed, shipped, delivered, and cancelled. Status transitions are validated at the application layer to prevent illegal transitions (e.g., jumping from pending_payment directly to shipped). The shippingAddress and billingAddress are embedded documents rather than separate references, since address data is rarely shared across orders and embedding eliminates an extra database round-trip during order retrieval.

Field	Type	Constraints	Description
orderNumber	String	unique, indexed	Readable order ID (e.g., ORD-20260812-AB3F)
userId	ObjectId	ref: User, indexed	Customer who placed the order
items	[Embedded]	required, min: 1	{productId, name, variant, qty, unitPrice, lineTotal}
subtotal	Number (int)	required	Sum of line totals in cents
discountAmount	Number (int)	default: 0	Coupon discount applied in cents
taxAmount	Number (int)	required	Calculated tax in cents
shippingCost	Number (int)	required	Shipping fee in cents
totalAmount	Number (int)	required	Final payable amount in cents
status	String (enum)	required, indexed	State machine: pending_payment to delivered
couponId	ObjectId	ref: Coupon	Applied coupon reference (if any)
paymentMethod	String	required	stripe | paypal
paymentIntentId	String	indexed	Stripe PaymentIntent ID for webhooks
shippingAddress	Embedded Doc	required	{name, line1, line2, city, state, zip, country}

Table 5. Orders Collection Schema

3.4 Inventory Collection

The Inventory collection implements distributed stock management across multiple warehouse locations. Each document represents the stock level for a specific product variant at a specific warehouse, enabling the system to route orders to the nearest warehouse with sufficient stock and to provide accurate stock availability information during product browsing. The reservedQuantity field is critical for preventing overselling during high-traffic events: when a customer begins checkout, the system atomically increments the reservedQuantity using a MongoDB findOneAndUpdate operation with conditional checks (availableQuantity minus reservedQuantity must be greater than or equal to the requested quantity). If the checkout is not completed within a configurable timeout period (default 15 minutes), a background cron job decrements the reservedQuantity to release the held stock back to the available pool.

Field	Type	Constraints	Description
productId	ObjectId	ref: Product, indexed	Product reference
variantSku	String	indexed	Variant SKU (null for single-variant)
warehouseId	ObjectId	ref: Warehouse, indexed	Warehouse location reference
availableQty	Number (int)	required, min: 0	Physical stock on shelves
reservedQty	Number (int)	default: 0, min: 0	Stock held during checkout (15-min TTL)
lowStockThreshold	Number (int)	default: 10	Alert trigger level for restocking
compound index	(productId, warehouseId, variantSku)	unique	Prevents duplicate inventory records

Table 6. Inventory Collection Schema

3.5 Additional Collections Overview

Beyond the four core collections detailed above, the platform requires ten additional supporting collections to model the complete e-commerce domain. The Vendors collection stores vendor business information including commission rates and bank account details for payout processing. The Warehouses collection tracks physical warehouse locations. The Coupons collection implements the dynamic discount engine with fields for discount type (percentage, fixed, BOGO), validity period, usage limits, and

applicable product or category restrictions. The Reviews collection stores customer feedback with star ratings, text content, image attachments, and a moderation status field that admin staff can toggle. The Transactions collection provides a financial audit trail recording every payment event with Stripe or PayPal, including webhook payloads for reconciliation.

The RefreshTokens collection manages JWT refresh token storage in Redis with TTL-based expiration, supporting the token rotation security pattern. The OrderStatusHistory collection maintains a chronological log of every status transition for each order, enabling the real-time tracking timeline. The Categories collection defines the hierarchical category tree. The Notifications collection stores in-app notification records with read/unread tracking. The AuditLogs collection records all administrative actions for compliance and debugging, with automatic TTL expiration after 90 days to prevent unbounded collection growth.

Collection	Key Fields	Purpose
Vendors	name, commissionRate, bankAccount, isActive	Multi-vendor management and payouts
Warehouses	name, address, contactEmail, isActive	Physical warehouse locations
Coupons	code, type, value, minOrder, maxUses, validFrom, validTo	Dynamic discount engine
Reviews	userId, productId, rating, text, images, moderationStatus	Customer feedback with moderation
Transactions	orderId, provider, providerId, amount, status, webhookPayload	Financial audit trail
RefreshTokens	userId, tokenHash, expiresAt (Redis TTL)	Secure refresh token storage
OrderStatusHistory	orderId, fromStatus, toStatus, updatedBy, note	Order status transition log
Categories	name, slug, parentId, level, image	Hierarchical category tree
Notifications	userId, title, message, isRead, type	In-app notification inbox
AuditLogs	userId, action, entity, entityId, changes (TTL 90d)	Admin action audit trail

Table 7. *Additional Supporting Collections*

4. REST API Design

The REST API follows resource-oriented design principles with consistent URL naming conventions, proper HTTP method semantics (GET for retrieval, POST for creation, PUT/PATCH for updates,

DELETE for removal), and standardized response envelopes. All API responses follow a uniform JSON structure with success, data, message, and pagination fields, enabling the frontend to handle responses generically through Axios interceptors. Authentication is enforced through a two-middleware chain: the first middleware extracts and verifies the JWT access token from HTTP-only cookies, attaching the decoded user payload to the request object; the second middleware checks the user role against a required roles array for protected endpoints. Rate limiting is implemented using Redis-based sliding window counters to protect against brute-force attacks.

4.1 Authentication Endpoints

The authentication module implements a secure JWT-based flow with access and refresh token pairs. Access tokens have a short expiration (15 minutes) and are stored in HTTP-only cookies to prevent XSS extraction, while refresh tokens have a longer expiration (7 days) and are stored both in HTTP-only cookies and in a Redis hash for server-side validation and revocation capability. The token rotation pattern ensures that every time a refresh token is used to obtain a new access token, a new refresh token is also issued and the old one is invalidated. Registration endpoints include role-specific validation: vendor manager registration requires a valid vendorId, and warehouse staff registration requires a valid warehouseId.

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user (role-specific validation)
POST	/api/auth/login	Public	Authenticate and set HTTP-only cookies
POST	/api/auth/refresh	Cookie-based	Rotate refresh token, issue new pair
POST	/api/auth/logout	Authenticated	Clear cookies, invalidate token
GET	/api/auth/me	Authenticated	Return current user profile
PATCH	/api/auth/password	Authenticated	Change password (verify current)

Table 8. *Authentication API Endpoints*

4.2 Product Catalog Endpoints

The product catalog API provides public browsing endpoints and vendor-scoped management endpoints. The search endpoint accepts query parameters for text, category, subcategory, price range, minimum rating, and stock status, all processed through a MongoDB aggregation pipeline that combines $match,

$lookup, $addFields, and $facet for simultaneous data and count retrieval. Text search leverages a pre-built MongoDB text index on name, description, and brand fields, returning results sorted by text relevance score. Product creation and update endpoints enforce vendor-level scoping: a vendor manager can only create or modify products belonging to their own vendorId.

Method	Endpoint	Access	Description
GET	/api/products	Public	Search, filter, sort, paginate products
GET	/api/products/:id	Public	Single product with reviews summary
GET	/api/products/categories	Public	Category tree for navigation
POST	/api/products	Vendor Mgr	Create product (vendor-scoped)
PUT	/api/products/:id	Vendor (own)	Update product details and images
DELETE	/api/products/:id	Vendor/Admin	Soft-delete (isActive = false)

Table 9. *Product Catalog API Endpoints*

4.3 Orders and Checkout Endpoints

The orders API implements a two-phase checkout process separating order creation from payment confirmation. In the first phase, the client submits a checkout request; the server validates stock using atomic findOneAndUpdate within a MongoDB session, calculates pricing, applies coupons, creates the order in pending_payment status, and returns a Stripe client secret. In the second phase, after the client confirms payment on the frontend, a Stripe webhook notifies the server, which transitions the order to processing, permanently decrements inventory, generates the invoice PDF, and emits a Socket.io event.

Method	Endpoint	Access	Description
POST	/api/orders/checkout	Customer	Create order, reserve stock, return payment intent
POST	/api/webhooks/stripe	Stripe (sig)	Handle payment events asynchronously
GET	/api/orders	Customer/Admin	List orders with pagination and filters
GET	/api/orders/:id	Customer/Admin	Order detail with status history

PATCH	/api/orders/:id/status	Warehouse/Admin	Update status (state machine validated)
GET	/api/orders/:id/invoice	Customer (own)	Download PDF invoice

Table 10. Orders and Checkout API Endpoints

4.4 Inventory, Coupons, and Reviews Endpoints

The inventory API provides warehouse staff and admins endpoints to view stock levels, adjust quantities, and transfer stock between warehouses. The coupons API supports full CRUD for admins, a validation endpoint for the checkout service, and a public listing of active coupons. The reviews API allows verified purchasers to submit reviews (preventing duplicates via compound unique index on userId and productId), and provides admins with a moderation endpoint.

Method	Endpoint	Access	Description
GET	/api/inventory	Warehouse/Admin	Stock levels across warehouses
PATCH	/api/inventory/:id	Warehouse Staff	Adjust availableQty
POST	/api/inventory/transfer	Admin	Move stock between warehouses
POST	/api/coupons	Admin	Create new coupon/discount rule
GET	/api/coupons/active	Public	List active promotions
POST	/api/coupons/:code/validate	Customer	Validate coupon and compute discount
POST	/api/reviews	Customer (verified)	Submit review (purchase verified)
GET	/api/products/:id/reviews	Public	Paginated reviews for a product
PATCH	/api/reviews/:id/moderate	Admin	Approve, reject, or remove review

Table 11. Inventory, Coupons, and Reviews Endpoints

5. Core Feature Technical Deep Dives

5.1 Multi-Vendor Role-Based Access Control

The RBAC system implements four distinct authorization tiers with granular permission sets controlling API endpoint access, frontend route visibility, and data scoping. The Super Admin role has unrestricted access to all system resources. Vendor Managers are scoped to their own vendor entity and can manage products and view orders for their products. Warehouse Staff can view and adjust inventory at their assigned warehouse and update order fulfillment status. Customers can browse the catalog, manage their own orders, submit reviews, and apply coupons. The implementation uses a custom Express middleware that accepts an array of allowed roles, extracts the user role from the decoded JWT, and returns 403 if unauthorized. This middleware composes with the authentication middleware for a protected route chain.

The authentication flow begins with user registration, where the password is hashed using bcrypt with 12 salt rounds. Upon login, the server generates two JWTs: an access token (15-minute expiry, contains userId, email, role) and a refresh token (7-day expiry, contains only userId). Both are set as HTTP-only, Secure, SameSite=Strict cookies. The refresh token hash is stored in Redis with matching TTL for server-side revocation. When the access token expires, the frontend calls the refresh endpoint; the server validates, issues a new token pair, invalidates the old refresh token (rotation), and sets new cookies. This rotation ensures compromised tokens are useful only once, significantly reducing the exploitation window.

Role	Products	Orders	Inventory	Admin
Super Admin	Full CRUD all	All orders	All warehouses	Users, config
Vendor Mgr	Own vendor CRUD	Own vendor orders	Read-only	Own profile
Warehouse	Read-only	Fulfillment status	Own warehouse	None
Customer	Browse	Own orders	None	None

Table 12. *Role Permission Matrix*

5.2 Distributed Inventory and Stock Reservation

The inventory management system addresses the critical challenge of preventing overselling during concurrent checkout sessions. The implementation uses MongoDB sessions to execute multi-document transactions that atomically reserve stock across multiple inventory records. When a customer initiates checkout, the system starts a session, begins a transaction, and for each line item, executes a findOneAndUpdate on the Inventory document with a condition that (availableQty minus reservedQty) is greater than or equal to the requested quantity, incrementing reservedQty. If any reservation fails, the entire transaction is aborted and rolled back, ensuring consistent inventory state.

A background cron job (node-cron) runs every minute to scan for expired stock reservations. It queries Inventory documents where reservedQty > 0, cross-references associated pending orders older than 15 minutes, and decrements reservedQty to release held stock. This timeout approach handles edge cases

like network disconnections. For real-time inventory sync, Socket.io emits inventory update events whenever stock levels change. Connected warehouse dashboards subscribe to these events and update their local state, ensuring consistent stock information across all operators within milliseconds.

5.3 Stripe and PayPal Payment Integration

Payment processing follows a secure server-side integration pattern. When the client initiates checkout, the backend creates a Stripe PaymentIntent with the calculated order amount and returns the client secret. The frontend uses Stripe.js to collect card details and confirm the payment, passing only the client secret to Stripe servers. Upon success, Stripe sends a webhook (payment_intent.succeeded) to the backend, which verifies the webhook signature using the Stripe webhook secret, looks up the corresponding order, transitions status to processing, permanently decrements reserved inventory, generates the invoice PDF, and emits a Socket.io event to notify the customer.

Webhook verification prevents forged payment confirmations. The implementation extracts the Stripe-Signature header, constructs the expected signature using the webhook secret and raw request body, and compares using a timing-safe function to prevent timing attacks. The webhook endpoint is idempotent: receiving the same event multiple times does not cause duplicate processing, achieved by checking current order status before transitioning. The Transaction collection records every webhook event with its complete payload for financial reconciliation. PayPal integration follows a similar webhook flow using the PayPal Webhook API with the order ID stored in PayPal order metadata for correlation.

5.4 Real-Time Order Tracking and Status Pipeline

The real-time order tracking system uses Socket.io to provide customers with live updates as their order progresses through the fulfillment pipeline. The order status follows a strict state machine: pending_payment transitions to processing or cancelled; processing transitions to confirmed, shipped, or cancelled; shipped transitions to delivered. Every transition creates an OrderStatusHistory document recording the from/to statuses, the user, an optional note (tracking number, cancellation reason), and a timestamp. The frontend displays these entries as a vertical timeline.

Socket.io namespaces isolate event channels and prevent unauthorized access. Customers join a room scoped to their orderId and receive events only for their own orders. Warehouse staff join a warehouse-specific room and receive events for all orders assigned to their warehouse. When an admin or warehouse staff member updates an order status through the API, the controller emits a Socket.io event to the relevant rooms, and connected clients update their UI in real time without page refresh. The system also handles reconnection gracefully: when a client disconnects and reconnects (e.g., switching between Wi-Fi and mobile data), the server replays the latest order status so the client can resynchronize its state.

5.5 Dynamic Coupon and Discount Engine

The coupon engine supports four discount types: percentage-based (e.g., 20% off), fixed cart amount (e.g., $10 off the total), buy-one-get-one-free (BOGO, applied to the lowest-priced qualifying item), and free shipping. Each coupon document contains applicability rules including minimum order amount, maximum discount cap, allowed categories or specific product IDs, and a user restriction list (allowing targeted coupons for specific customer segments). The validation service is called during checkout and checks the coupon code against all applicable rules: validity period (current date must be within validFrom and validTo), usage count (current uses must not exceed maxUses), per-user limit (if specified), and cart eligibility (minimum order amount and product/category restrictions).

The discount calculation follows a specific precedence order: first, product-level sale prices are applied; then, the coupon discount is computed on the discounted subtotal; finally, tax is calculated on the amount after all discounts. This ensures that coupons do not apply to tax or shipping amounts. The BOGO implementation identifies the lowest-priced qualifying item in the cart and sets its line total to zero, effectively making it free. All coupon operations are logged in the AuditLogs collection with the coupon code, discount amount, and the order ID, enabling administrators to track coupon performance and detect potential abuse patterns such as multiple accounts using the same single-use coupon.

5.6 Reviews, Ratings, and Moderation

The review system allows verified purchasers (users with a completed order containing the product) to submit star ratings (1-5), text reviews (maximum 2000 characters), and up to 5 image attachments (uploaded to S3 via pre-signed URLs). Duplicate reviews are prevented by a compound unique index on userId and productId at the database level. When a review is submitted, the system updates the product averageRating field using a MongoDB aggregation pipeline that recalculates the mean rating across all approved reviews for the product, ensuring the displayed rating always reflects the current review corpus.

The moderation dashboard provides administrators with a queue of pending reviews, sorted by submission date. Each review can be approved (published to the product page), rejected (hidden from public view, with an optional reason sent to the reviewer via notification), or removed entirely. New reviews are initially placed in a pending_moderation status and only become visible on the product page after approval. This ensures that inappropriate content does not appear on live product pages. The review API supports pagination with configurable page size, sorting by most recent, highest rated, or most helpful (based on other users marking reviews as helpful), and filtering by star rating level.

5.7 Sales Analytics and Revenue Dashboard

The analytics module provides interactive data visualization powered by Recharts on the frontend, with data sourced from MongoDB aggregation pipelines executed on the backend. The dashboard displays four key metric cards: total revenue, total orders, average order value, and conversion rate (calculated as completed orders divided by total created orders). Below these cards, a series of charts visualize trends over time: a line chart showing daily revenue over the past 30 days, a bar chart comparing weekly order

volumes, a pie chart breaking down revenue by product category, and a horizontal bar chart listing the top 10 best-selling products by revenue contribution.

The backend analytics endpoint uses MongoDB aggregation with $match (date range filter), $group (daily/weekly/monthly buckets), $lookup (to enrich with product and category data), and $sort (for ranking). To optimize performance, the server caches aggregation results in Redis with a 5-minute TTL, since analytics data does not need to be real-time accurate. For larger datasets, the aggregation pipeline uses the $facet stage to run multiple aggregations in a single database round-trip, reducing latency. The dashboard also supports date range filtering (last 7 days, 30 days, 90 days, custom range) and vendor-level filtering (for vendor managers who should only see their own product performance metrics).

5.8 PDF Invoice and Packing Slip Generation

The platform automatically generates two types of PDF documents upon order confirmation: a customer-facing tax invoice and a warehouse-facing packing slip. Invoice generation uses a server-side PDF library (PDFKit or jsPDF) that accepts structured data (order details, line items, pricing breakdown, tax calculation, shipping address, and payment information) and renders a professionally formatted PDF with company branding, a unique invoice number, and a QR code containing the order number for quick lookup. The invoice PDF is stored in an S3 bucket and its URL is saved in the Orders collection for download access.

The packing slip contains a condensed version of the order information optimized for warehouse use: order number, customer name and address (for shipping label purposes), a table of items with quantities and SKU numbers, and a large barcode or QR code for warehouse scanning. Unlike the invoice, the packing slip does not include pricing information since it is handled by warehouse staff who do not need visibility into financial details. Both document types are generated asynchronously after the payment webhook confirms the order, using a queue-based approach where the PDF generation task is placed in a processing queue to avoid blocking the webhook response. Once generated, a notification is sent to the relevant parties: the customer receives an email with the invoice download link, and the warehouse dashboard displays the packing slip for the assigned order.

5.9 Automated Warehouse Fulfillment and Low-Stock Alerts

A node-cron based background job system runs on the server to monitor inventory levels across all warehouses and trigger automated alerts when stock falls below configurable thresholds. Every minute, a cron job queries the Inventory collection for documents where availableQty minus reservedQty is less than or equal to the lowStockThreshold field (default: 10 units). For each item found, the system checks whether an alert was already sent within the last 24 hours (by querying the Notifications collection) to prevent alert spam. If no recent alert exists, it creates a notification record for the warehouse manager and optionally sends an email via SendGrid to the configured supplier contact email associated with the product vendor.

The fulfillment workflow begins when an order transitions to the confirmed status. At this point, the system identifies the optimal warehouse for each line item based on proximity to the shipping address (using a simple distance calculation from warehouse coordinates to the shipping zip code centroid) and available stock levels. A fulfillment task is created for each warehouse, containing the list of items to pick, pack, and ship. Warehouse staff see these tasks in their dashboard, ordered by priority (express orders first, then by order creation time). When a staff member marks an item as packed, the system generates the packing slip PDF and updates the order status to shipped with the tracking number. Real-time inventory deduction happens at the point of shipping (not at order creation), which means the reservedQty is decremented and availableQty is permanently reduced.

6. Development Timeline and Phased Roadmap

The development roadmap is organized into six phases spanning 12 weeks, with each phase building upon the foundations laid by the previous one. The phasing strategy prioritizes features based on dependency ordering: authentication and user management must exist before any protected API can be built; the product catalog must exist before orders can be placed; and orders must exist before payment processing, real-time tracking, and analytics can be implemented. This dependency chain determines the phase sequence and ensures that each sprint delivers a functional increment that can be demonstrated and tested independently. Each phase includes specific deliverables, testing requirements, and a brief description of what the interviewer should look for during a placement demonstration.

Phase	Weeks	Features	Key Deliverables
1. Foundation	1-2	Auth (JWT + refresh), RBAC middleware, user CRUD, Docker setup, DB connection	Login/register flow, protected routes, role-scoped API access, Docker Compose running
2. Catalog	3-4	Product CRUD, text search, faceted filtering, image upload to S3, category tree	Searchable product grid with filters, vendor product management pages
3. Inventory	5-6	Inventory model, stock reservation with transactions, warehouse CRUD, low-stock alerts	Distributed stock levels, race-condition-safe checkout reservation, cron alerts
4. Payments	7-8	Stripe integration, webhook handling, checkout flow, coupon engine, invoice PDF gen	End-to-end checkout, payment confirmation, downloadable invoices
5. Real-Time	9-10	Socket.io order tracking, status pipeline, reviews/ratings/moderation, notifications	Live order status timeline, review submission, admin moderation queue
6. Analytics	11-12	Sales dashboard, Recharts visualizations, revenue reports, packing slips, deployment	Interactive analytics dashboard, production deployment on Render/AWS

Table 13. Phased Development Roadmap

6.1 Phase 1: Foundation (Weeks 1-2)

The foundation phase establishes the project scaffolding, development environment, and authentication system that all subsequent features depend on. The first week focuses on project initialization: creating the monorepo structure, setting up Docker Compose with MongoDB (replica set), Redis, and Node.js containers, configuring environment variables, establishing the Mongoose connection with retry logic, and implementing the Express server with helmet, CORS, and request logging middleware. The second week implements the complete authentication system including user registration with bcrypt password hashing, JWT access and refresh token generation, HTTP-only cookie management, the token rotation endpoint, and the RBAC authorization middleware. Unit tests for auth flows are written using Jest and Supertest, covering registration, login, token refresh, and role-based endpoint protection.

6.2 Phase 2: Product Catalog (Weeks 3-4)

The catalog phase builds the product browsing and management experience. Week 3 focuses on the backend: Mongoose Product model with all fields, text index creation for search, the product search endpoint using MongoDB aggregation pipelines with $match, $lookup (for vendor data), $addFields (for discount percentage calculation), and $facet (for simultaneous data and count), and the vendor-scoped CRUD endpoints with ownership validation. Week 4 builds the frontend: product listing page with infinite scroll pagination, sidebar filter panel (category, price range, rating, stock status) with URL query parameter synchronization, product detail page with image gallery and variant selector, and the vendor management page for creating and editing products with image upload to S3 via pre-signed URLs.

6.3 Phase 3: Inventory Management (Weeks 5-6)

The inventory phase implements the distributed stock management system that prevents overselling. Week 5 develops the Inventory model, the stock reservation logic using MongoDB sessions and findOneAndUpdate with conditional operators, and the reservation release cron job. Week 6 builds the warehouse management dashboard with real-time stock level display, the stock adjustment interface for warehouse staff, the stock transfer endpoint for administrators, and the low-stock alert system with email notifications via SendGrid. This phase also implements the integration between the checkout flow and the inventory reservation system, ensuring that the product catalog accurately reflects real-time stock availability.

6.4 Phase 4: Payments and Checkout (Weeks 7-8)

The payments phase connects the catalog and inventory systems to the revenue pipeline. Week 7 implements the Stripe integration: PaymentIntent creation on the backend, Stripe Elements card input on the frontend, the two-phase checkout flow (order creation with stock reservation, then payment confirmation), and the webhook endpoint with signature verification. Week 8 adds the coupon and discount engine (percentage, fixed, BOGO, free shipping), the order status state machine, the automated

PDF invoice generation, and the PayPal integration as a secondary payment method. End-to-end testing with Stripe test mode ensures the complete purchase flow works correctly from product browsing to invoice download.

6.5 Phase 5: Real-Time Features (Weeks 9-10)

The real-time phase adds live interactivity and user engagement features. Week 9 implements the Socket.io server with namespaced channels for orders, inventory, and admin notifications, the real-time order tracking timeline on the customer order detail page, and the warehouse fulfillment dashboard with live status updates. Week 10 builds the review and rating system (submission, display, star rating aggregation, image attachments), the admin moderation queue with approve/reject workflows, the in-app notification system, and the review deduplication logic. Integration testing ensures that Socket.io events are correctly emitted and received across multiple connected clients.

6.6 Phase 6: Analytics and Deployment (Weeks 11-12)

The final phase focuses on business intelligence and production readiness. Week 11 builds the sales analytics dashboard with Recharts visualizations: revenue line chart, order volume bar chart, category revenue pie chart, and top products ranking. The backend implements MongoDB aggregation pipelines with Redis caching for performance. Week 12 focuses on deployment: configuring the production Docker Compose with Nginx reverse proxy, setting up CI/CD with GitHub Actions, deploying to Render or AWS Elastic Beanstalk, configuring environment variables securely, running load tests to identify performance bottlenecks, and writing comprehensive README documentation with architecture diagrams, ER diagrams, API docs, and local setup instructions using Docker Compose.

7. Environment Configuration

Environment variables are managed through a .env file (with .env.example committed to version control as a template). All sensitive configuration values (database URIs, API keys, JWT secrets) are excluded from git via .gitignore. The following table lists all required and optional environment variables organized by service. During development, these values point to local Docker containers; in production, they point to cloud-managed services such as MongoDB Atlas, AWS ElastiCache for Redis, and the live Stripe API endpoint. The frontend .env file contains only the VITE_API_BASE_URL variable, keeping all sensitive configuration on the backend.

Variable	Service	Example Value	Description
PORT	Server	5000	Express server port
MONGODB_URI	MongoDB	mongodb://mongo:27017/omnichannel?replicaSet=rs0	MongoDB connection with replica set

REDIS_URL	Redis	redis://redis:6379	Redis connection URL
JWT_ACCESS_SECRET	Auth	(64-char random string)	Access token signing secret
JWT_REFRESH_SECRET	Auth	(64-char random string)	Refresh token signing secret
STRIPE_SECRET_KEY	Stripe	sk_live_...	Stripe API secret key
STRIPE_WEBHOOK_SECRET	Stripe	whsec_...	Stripe webhook verification secret
SENDGRID_API_KEY	SendGrid	SG.xxxxx	SendGrid email API key
AWS_S3_BUCKET	AWS S3	omnichannel-assets	S3 bucket for images and PDFs
NODE_ENV	General	development | production	Application environment mode

Table 14. *Environment Variables Reference*

8. Testing and Quality Strategy

A robust testing strategy is essential for demonstrating production-readiness during placement interviews. The testing pyramid for this project consists of three layers: unit tests for individual functions and utility modules, integration tests for API endpoints that exercise the full request-response cycle including database operations, and end-to-end tests using a headless browser (Playwright or Cypress) that verify complete user workflows such as registration, product browsing, checkout, and order tracking. The testing stack uses Jest as the test runner, Supertest for HTTP assertion against Express endpoints, and MongoDB Memory Server for an in-memory MongoDB instance that provides fast, isolated test database operations without requiring a running MongoDB container.

Unit tests cover all service-layer functions (discount calculation, stock reservation logic, coupon validation) and utility functions (PDF generation helpers, email formatting). Integration tests verify each API endpoint with authenticated requests, testing both success cases and error cases (invalid input, insufficient permissions, not-found resources, concurrent stock conflicts). A critical integration test suite verifies the checkout transaction: it creates an order with multiple items, verifies stock reservation, simulates a payment webhook, confirms inventory deduction, and checks invoice generation, all within a single test that exercises the complete order lifecycle. Error-handling middleware is tested to ensure that all error responses follow the standard envelope format, even for unexpected server errors, providing a clean API contract for frontend consumption.

9. Deployment Strategy

The deployment strategy targets cloud platforms suitable for placement portfolio demonstration, with primary support for Render (simplest setup) and AWS (most impressive for interviews). The Docker Compose configuration defines four services: the React frontend (built with Vite and served by Nginx), the Express backend, MongoDB (with replica set initialization script), and Redis. For Render deployment, the frontend is deployed as a static site, the backend as a web service, and MongoDB and Redis are provisioned as managed add-ons. For AWS deployment, the application uses Elastic Beanstalk for the backend, S3 + CloudFront for the frontend, DocumentDB or Atlas for MongoDB, and ElastiCache for Redis.

CI/CD is implemented through GitHub Actions with two workflows: a test workflow that runs on every pull request (executing the full Jest test suite, linting with ESLint, and type-checking with TypeScript), and a deploy workflow that runs on push to the main branch (building Docker images, pushing to a container registry, and deploying to the target platform). Environment variables are configured as platform secrets, never committed to the repository. The production Nginx configuration includes security headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options), gzip compression, and rate limiting. A health check endpoint (/api/health) returns the status of all dependencies (MongoDB connection, Redis connection, memory usage) and is used by the platform load balancer to determine service availability.